



Universidad Tecnológica Nacional (FRGP)

Trabajo Práctico Integrador

Base de datos II

Grupo N°15

Jazmín Nahir Erasun Sgur - Leg: 32509

Raul Oscar Luppini - Leg: 30439

Matías Hernán Ramírez - Leg: 26078

Germán Gabriel Cortés - Leg: 27652

Docentes

Ángel Simón

Laura Vélez

Abel Faure

Brian Lara Campos

Descripción del Sistema – Ecommerce

El sistema desarrollado corresponde a una plataforma de comercio electrónico que permite gestionar de manera integral los procesos de venta online. Su propósito es brindar una solución eficiente para la administración de productos, usuarios y pedidos, garantizando la integridad de los datos y la trazabilidad de las operaciones.

Objetivos principales:

1. **Gestión de usuarios:** registrar y administrar clientes y administradores, incluyendo datos personales, credenciales de acceso y roles.
2. **Gestión de productos:** mantener actualizado el catálogo de productos con información de marca, categoría, descripción, precios y stock disponible.
3. **Gestión de pedidos:** registrar pedidos realizados por los clientes, calcular el precio total, controlar el estado (pendiente, confirmado, cancelado) y el método de pago.
4. **Detalle de pedidos:** almacenar los productos asociados a cada pedido, con cantidad, precio unitario y descuentos aplicados.
5. **Control de stock:** actualizar automáticamente el stock de productos al registrar ventas o cancelaciones.
Adicionalmente, generar alertas de reposición cuando el inventario alcance el nivel de stock mínimo configurado por el Administrador del e-commerce.
6. **Historial y consultas:** permitir consultas sobre pedidos realizados, productos disponibles y clientes frecuentes.
7. **Auditoría:** Registrar de forma automática e histórica las modificaciones críticas del sistema, como por ejemplo, los cambios en los precios de venta.

Alcance del sistema:

- El sistema está diseñado para cubrir las operaciones básicas de un ecommerce, asegurando:
- Integridad referencial mediante claves primarias y foráneas.
- Restricciones de validación (ej. cantidad de productos > 0, estado de usuario activo/inactivo).

- Automatización de procesos críticos mediante procedimientos almacenados y triggers.
- Reportes y vistas (ej. reporte para reposición de stock y ranking de ventas).
- Uso de bajas lógicas para la eliminación de registros (usuarios, productos, pedidos) con el fin de proteger la integridad referencial y el historial del sistema.

Principales datos que maneja:

- **Usuarios:** información personal, credenciales y rol.
- **Productos:** código, nombre, marca, categoría, precios, stock.
- **Pedidos:** cliente asociado, fecha, estado, método de pago, precio total.
- **Detalle de pedidos:** productos incluidos, cantidades y precios.
- **Imágenes:** URLs asociadas a productos para su visualización en el catálogo.

Diagrama de Entidad Relación Inicial

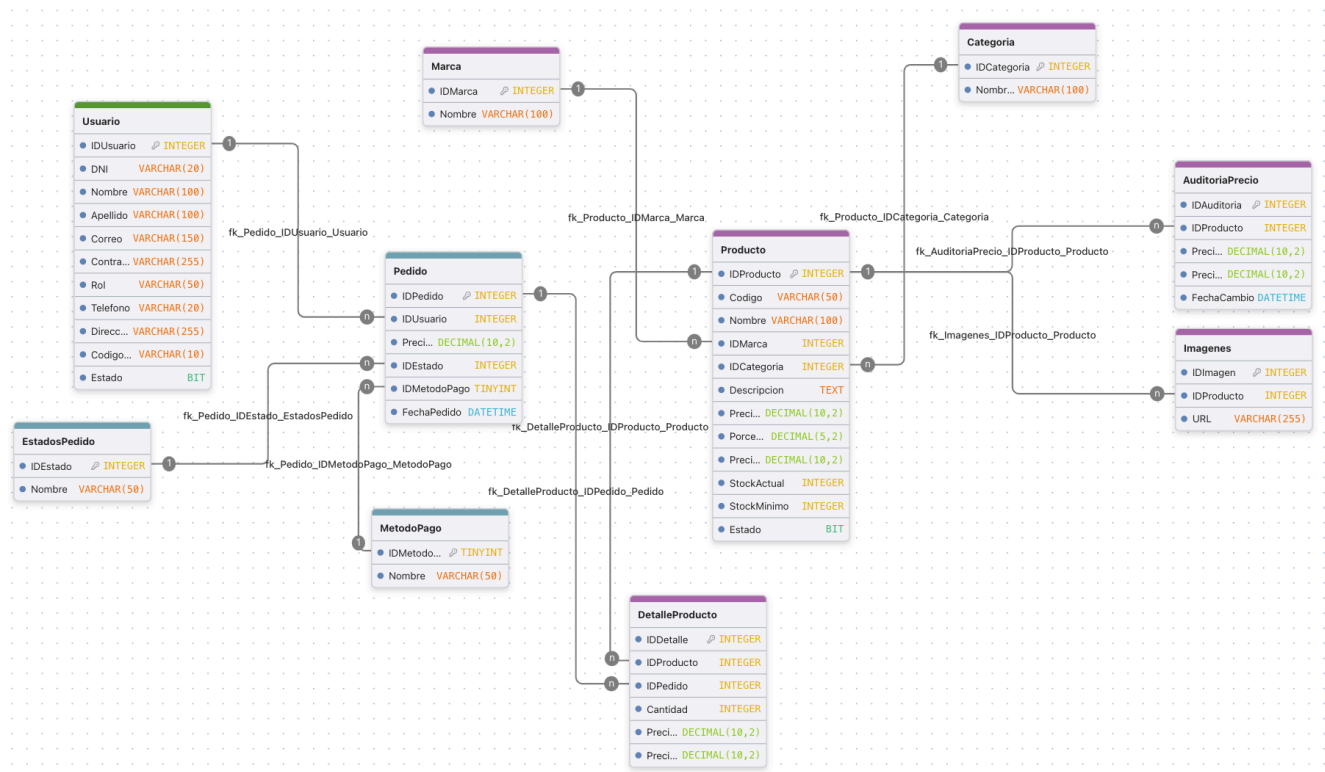
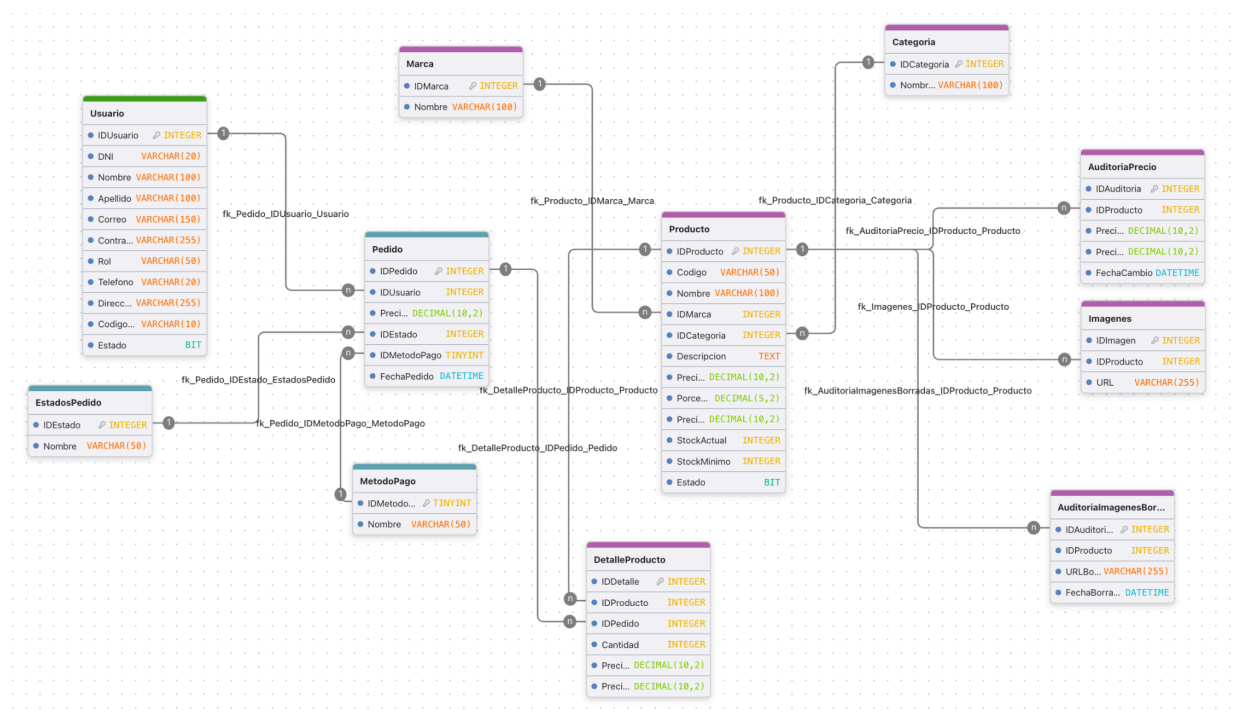


Diagrama de Entidad Relación actualizado tras desarrollo del TP



Objetos de Base de Datos clave en el sistema

Vista 1

Se utiliza **VW_ListarProductos** para consultar de manera unificada la información de los productos junto con sus marcas y categorías. Su objetivo es simplificar las consultas en la aplicación, evitando tener que escribir múltiples JOIN cada vez que se necesita mostrar productos.

```
CREATE VIEW [dbo].[VW_ListarProductos] AS
SELECT
    p.IDProducto,
    p.Codigo,
    p.Nombre,
    m.IDMarca AS MarcaId,
    m.Nombre AS MarcaNombre,
    c.IDCategoria AS CategoriaId,
    c.Nombre AS CategoriaNombre,
    p.Descripcion,
    p.PrecioCompra,
    p.PorcentajeGanancia,
    p.PrecioVenta,
    p.StockActual,
    p.StockMinimo,
    p.Estado
FROM Producto p
INNER JOIN Marca m ON p.IDMarca = m.IDMarca
INNER JOIN Categoria c ON p.IDCategoria = c.IDCategoria;

GO
```

Vista 2

Se utiliza **VW_ProductosAReponer** para consultar en forma unificada cuáles son los productos se encuentran con stock bajo (pudiendo visualizar la información de marcas, categorías y el cálculo de stock faltante). El objetivo principal de esta vista es simplificar las consultas en la aplicación, evitando tener que calcular la diferencia de stock manualmente o escribir múltiples JOIN cada vez que el administrador necesite hacer un chequeo de stock faltante.

```
CREATE VIEW VW_ProductosAReponer AS
SELECT
    p.IDProducto,
    p.Codigo,
    p.Nombre AS Producto,
    m.Nombre AS Marca,
    c.Nombre AS Categoria,
    p.StockActual,
    p.StockMinimo,
    (p.StockMinimo - p.StockActual) AS CantidadAReponer
FROM Producto p
INNER JOIN Marca m ON p.IDMarca = m.IDMarca
INNER JOIN Categoria c ON p.IDCategoria = c.IDCategoria
WHERE p.StockActual <= p.StockMinimo
AND p.Estado = 1;

GO
```

Vista 3

Se utiliza **VW_ClientesFrecuentes** para consultar en forma unificada información sobre la actividad de compra de los clientes, mostrando la cantidad de pedidos realizados y el monto total gastado por cada usuario. El objetivo principal de esta vista es facilitar la generación de reportes comerciales y el análisis de clientes frecuentes, evitando tener que realizar cálculos de agregación (**COUNT** y **SUM**) y múltiples uniones entre tablas cada vez que se requiera esta información en la aplicación.

```
CREATE VIEW VW_ClientesFrecuentes AS
SELECT
    u.IDUsuario,
    u.Nombre,
    u.Apellido,
    u.Correo,
    COUNT(p.IDPedido) AS CantidadPedidos,
    SUM(p.Precio) AS TotalGastado
FROM Usuario U
INNER JOIN Pedido P
    ON u.IDUsuario = p.IDUsuario
GROUP BY
    u.IDUsuario,
    u.Nombre,
    u.Apellido,
    u.Correo;

GO
```

Vista 4

Se utiliza **VW_ProductoMasVendido** para obtener la cantidad de unidades vendidas de cada producto, mostrando el ID, el nombre y el total de las ventas realizadas. El objetivo principal de esta vista es facilitar la obtención de la información sobre los productos con mayor demanda, y de esta manera evitar la utilización de cálculos de agregación y uniones entre tablas cada vez que se requiera la información.

```
CREATE VIEW VW_ProductoMasVendido as
Select
P.IDProducto,
P.Nombre,
SUM (DP.Cantidad) as Venta_Total

from Producto as P
inner join DetalleProducto as DP
on DP.IDProducto = P.IDProducto
group by
P.IDProducto,
P.Nombre;

GO
```


Procedimiento Almacenado 1

El procedimiento **sp_InsertarProducto** se utiliza para registrar un nuevo producto en la base de datos. Su función principal es insertar un registro en la tabla Producto con todos los atributos necesarios para que el sistema de ecommerce pueda gestionarlo correctamente.

```
CREATE PROCEDURE [dbo].[sp_InsertarProducto]
    @Codigo VARCHAR(50),
    @Nombre VARCHAR(100),
    @IDMarca INT,
    @IDCategoria INT,
    @Descripcion NVARCHAR(255),
    @PrecioCompra DECIMAL(10,2),
    @PorcentajeGanancia DECIMAL(5,2),
    @PrecioVenta DECIMAL(10,2),
    @StockActual INT,
    @StockMinimo INT,
    @Estado BIT
AS
BEGIN
    INSERT INTO Producto (Codigo, Nombre, IDMarca, IDCategoria,
        Descripcion, PrecioCompra, PorcentajeGanancia, PrecioVenta, StockActual,
        StockMinimo, Estado)
        VALUES (@Codigo, @Nombre, @IDMarca, @IDCategoria, @Descripcion,
            @PrecioCompra, @PorcentajeGanancia, @PrecioVenta, @StockActual,
            @StockMinimo, @Estado);

    SELECT SCOPE_IDENTITY() AS IDGenerado;
END;
```

Procedimiento Almacenado 2

Se utiliza **sp_DetallePedido** para obtener el detalle completo de un pedido específico, incluyendo los datos del cliente, los productos adquiridos, las cantidades, los precios unitarios y el subtotal correspondiente a cada producto. El objetivo principal de este procedimiento es centralizar la lógica de consulta de pedidos en la base de datos, simplificando el acceso a la información y evitando la necesidad de realizar múltiples consultas y uniones entre tablas para reconstruir el detalle de una compra.

```
CREATE PROCEDURE sp_DetallePedido
    @IDPedido INT
AS
BEGIN
    SELECT
        p.IDPedido,
        p.FechaPedido,
        u.Nombre + ' ' + u.Apellido AS Cliente,
        pr.Nombre AS Producto,
        dp.Cantidad,
        dp.PrecioUnitario,
        dp.Cantidad * dp.PrecioUnitario AS Subtotal
    FROM Pedido p
    INNER JOIN Usuario u
        ON p.IDUsuario = u.IDUsuario
    INNER JOIN DetalleProducto dp
        ON p.IDPedido = dp.IDPedido
    INNER JOIN Producto pr
        ON dp.IDProducto = pr.IDProducto
    WHERE p.IDPedido = @IDPedido;
END;
GO
```

Trigger 1

Se implementó un trigger de auditoría en la tabla Producto que se dispara automáticamente después de una actualización. El objetivo es registrar en la tabla AuditoriaPrecio cualquier modificación en el precio de venta de un producto. De esta manera, se conserva un historial con el precio anterior, el nuevo precio y la fecha del cambio, lo que permite garantizar la trazabilidad de las modificaciones realizadas.

```
CREATE TRIGGER trg_AuditoriaPrecio
ON Producto
AFTER UPDATE
AS
BEGIN
    INSERT INTO AuditoriaPrecio (IDProducto, PrecioAnterior, PrecioNuevo,
FechaCambio)
    SELECT d.IDProducto, d.PrecioVenta, i.PrecioVenta, GETDATE()
    FROM deleted d
    JOIN inserted i ON d.IDProducto = i.IDProducto
    WHERE d.PrecioVenta <> i.PrecioVenta;
END;

GO
```

Trigger 2

El trigger **trg_RestaurarStock** se desencadena cuando un pedido es actualizado (cancelado) y por ende los productos se eliminan (en forma lógica) del mismo, devolviendo las unidades correspondientes al inventario. Su objetivo es asegurar que el stock se mantenga siempre actualizado en la base de datos, evitando de esta manera que la aplicación tenga esta responsabilidad en el caso de que un cliente cancele una compra, asegurando la integridad de datos.

```
CREATE TRIGGER trg_RestaurarStock
ON Pedido
AFTER UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN deleted d ON i.IDPedido = d.IDPedido
        JOIN EstadoPedido es ON i.IDEstado = es.IDEstado
        WHERE es.Nombre = 'Cancelado' AND d.IDEstado <> i.IDEstado
    )

    BEGIN
        UPDATE p
        SET p.StockActual = p.StockActual + d.Cantidad
        FROM Producto p
        INNER JOIN DetalleProducto d ON p.IDProducto = d.IDProducto
        INNER JOIN inserted i ON d.IDPedido = i.IDPedido
        WHERE i.IDEstado = (SELECT IDEstado FROM EstadoPedido WHERE Nombre = 'Cancelado');

    END

END;
GO
```

Trigger 3

El trigger **TR_DescontarDeStock** se desencadena cuando se registra un nuevo detalle de producto, descontando de forma automática del inventario la cantidad de unidades vendidas. El objetivo de este trigger es asegurar que el stock se mantenga actualizado, evitando que la aplicación tenga esa responsabilidad y manteniendo la integridad de los datos.

```
Create trigger TR_DescontarDeStock
ON DetalleProducto
after insert
as
begin
    if exists (
        select 1
        from Producto P
        inner join Inserted I
            on P.IDProducto = I.IDProducto
        where P.StockActual < I.Cantidad
    )
    Begin
        Raiserror('Stock insuficiente para realizar la venta.',16,1);
        Rollback transaction;
        return;
    end;

Update P
set P.StockActual = P.StockActual - I.Cantidad
from Producto as P
inner join inserted I
    ON P.IDProducto = I.IDProducto;
end;

GO
```

Trigger 4

El trigger **trg_RespaldoImagenEliminada** se ejecuta AFTER DELETE, y en este caso, se desencadena cuando una imagen es eliminada.

Su función principal es capturar la URL de la imagen, el ID del producto asociado, la fecha y hora de borrado para luego poder consultarlo en la tabla AuditoriaImagenesBorradas.

```
CREATE TRIGGER trg_RespaldoImagenEliminada
ON Imagenes
AFTER DELETE
AS
BEGIN
    INSERT INTO AuditoriaImagenesBorradas (IDProducto, URLBorrada,
FechaBorrado)
    SELECT d.IDProducto, d.URL, GETDATE()
    FROM deleted d;
END;
GO
```

Links a los recursos:

- [Link a la demo](#)
- [Link a la documentación en nuestro repositorio](#)
- [Link general del repositorio](#)